

Just for Squares

Eigendecomposition is defined only for square matrices. It is not possible to eigendecompose an $M \times N$ matrix unless $M = N$. Nonsquare matrices can be decomposed using the SVD. Every square matrix of size $M \times M$ has M eigenvalues (scalars) and M corresponding eigenvectors. The purpose of eigendecomposition is to reveal those M vector-scalar pairs.

Interpretations of Eigenvalues and Eigenvectors

There are several ways of interpreting eigenvalues/vectors that I will describe in the next sections. Of course, the math is the same regardless, but having multiple perspectives can facilitate intuition, which in turn will help you understand how and why eigendecomposition is important in data science.

Geometry

I've actually already introduced you to the geometric concept of eigenvectors in [Chapter 5](#). In [Figure 5-5](#), we discovered that there was a special combination of a matrix and a vector such that the matrix *stretched*—but did not *rotate*—that vector. That vector is an eigenvector of the matrix, and the amount of stretching is the eigenvalue.

[Figure 13-1](#) shows vectors before and after multiplication by a 2×2 matrix. The two vectors in the left plot ($\mathbf{v}_1$ and $\mathbf{v}_2$) are eigenvectors whereas the two vectors in the right plot are not. The eigenvectors point in the same direction before and after postmultiplying the matrix. The eigenvalues encode the amount of stretching; try to guess the eigenvalues based on visual inspection of the plot. The answers are in the footnote.¹

That's the geometric picture: an eigenvector means that matrix-vector multiplication acts like scalar-vector multiplication. Let's see if we can write that down in an equation (we can, and it's printed in [Equation 13-1](#)).

Equation 13-1. Eigenvalue equation

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$$

¹ Approximately -0.6 and 1.6 .

Be careful with interpreting that equation: it does not say that the matrix *equals* the scalar; it says that the *effect* of the matrix on the vector is the same as the *effect* of the scalar on that same vector.

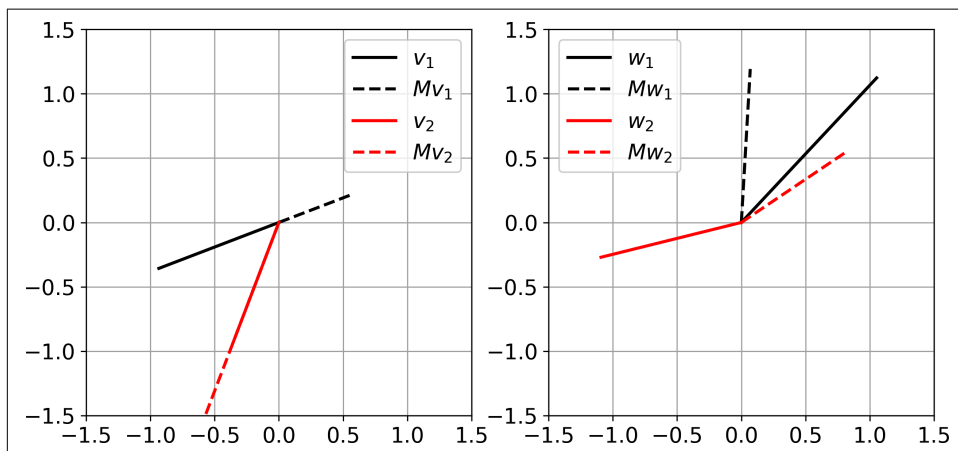


Figure 13-1. Geometry of eigenvectors

This is called the *eigenvalue equation*, and it's another key formula in linear algebra that is worth memorizing. You'll see it throughout this chapter, you'll see a slight variation of it in the following chapter, and you'll see it many times when learning about multivariate statistics, signal processing, optimization, graph theory, and a myriad of other applications where patterns are identified across multiple simultaneously recorded features.

Statistics (Principal Components Analysis)

One of the reasons why people apply statistics is to identify and quantify relationships between variables. For example, the rise of global temperatures correlates with the decline in the number of pirates,² but how strong is that relationship? Of course, when you have only two variables, a simple correlation (like what you learned in [Chapter 4](#)) is sufficient. But in a multivariate dataset that includes dozens or hundreds of variables, bivariate correlations cannot reveal global patterns.

Let's make this more concrete with an example. Cryptocurrencies are digital stores of value that are encoded in a blockchain, which is a system for keeping track of transactions. You've probably heard of Bitcoin and Ethereum; there are tens of thousands of other cryptocurrencies that have various purposes. We can ask whether the entirety of the cryptospace operates as a single system (meaning that the value of all coins go

² "Open Letter to Kansas School Board," Church of the Flying Spaghetti Monster, spaghettimonster.org/about/open-letter.

up and down together), or whether there are independent subcategories within that space (meaning that some coins or groups of coins change in value independently of the value of other coins).

We can test this hypothesis by performing a principal components analysis on a dataset that contains the prices of various cryptocurrencies over time. If the entire cryptomarket operates as a single entity, then the *scree plot* (a graph of the eigenvalues of the dataset's covariance matrix) would reveal that one component accounts for most of the variance of the system, and all other components account for very little variance (graph A in [Figure 13-2](#)). In contrast, if the cryptomarket had, say, three major subcategories with independent price movements, then we would expect to see three large eigenvalues (graph B in [Figure 13-2](#)).

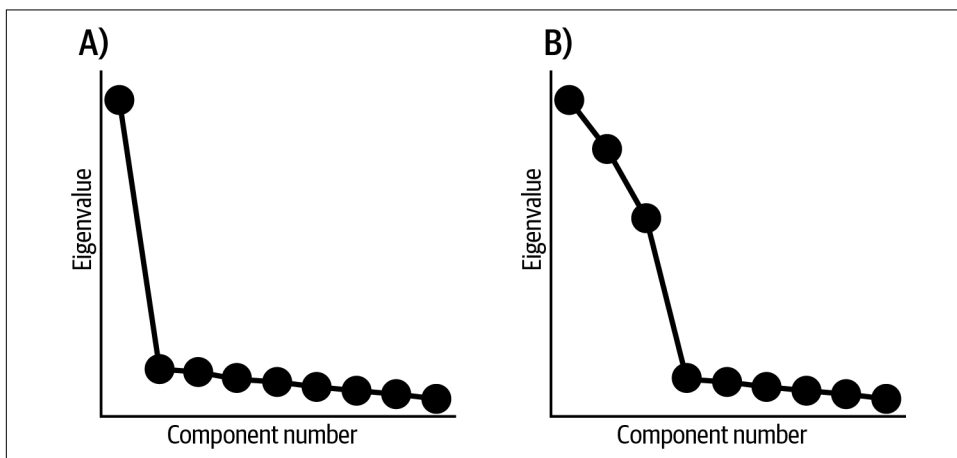


Figure 13-2. Simulated scree plots of multivariate datasets (data is simulated to illustrate outcome possibilities)

Noise Reduction

Most datasets contain noise. *Noise* refers to variance in a dataset that is either unexplained (e.g., random variation) or unwanted (e.g., electrical line noise artifacts in radio signals). There are many ways to attenuate or eliminate noise, and the optimal noise-reduction strategy depends on the nature and origin of the noise and on the characteristics of the signal.

One method of reducing random noise is to identify the eigenvalues and eigenvectors of a system, and “project out” directions in the data space associated with small eigenvalues. The assumption is that random noise makes a relatively small contribution to the total variance. “Projecting out” a data dimension means to reconstruct a dataset after setting some eigenvalues that are below some threshold to zero.

You’ll see an example of using eigendecomposition to reduce noise in [Chapter 15](#).

Dimension Reduction (Data Compression)

Information communications technologies like phones, internet, and TV create and transmit a huge amount of data, such as pictures and videos. Transmitting data can be time-consuming and expensive, and it is beneficial to *compress* the data before transmitting it. Compression means to reduce the size of the data (in terms of bytes) while having minimal impact on the quality of the data. For example, a TIFF format image file might be 10 MB, while the JPG converted version might be .1 MB while still retaining reasonably good quality.

One way to dimension-reduce a dataset is to take its eigendecomposition, drop the eigenvalues and eigenvectors associated with small directions in the data space, and then transmit only the relatively larger eigenvector/value pairs. It is actually more common to use the SVD for data compression (and you'll see an example in [Chapter 15](#)), although the principle is the same.

Modern data compression algorithms are actually faster and more efficient than the method previously described, but the idea is the same: decompose the dataset into a set of basis vectors that capture the most important features of the data, and then reconstruct a high-quality version of the original data.

Finding Eigenvalues

To eigendecompose a square matrix, you first find the eigenvalues, and then use each eigenvalue to find its corresponding eigenvector. The eigenvalues are like keys that you insert into the matrix to unlock the mystical eigenvector.

Finding the eigenvalues of a matrix is super easy in Python:

```
matrix = np.array([
    [1,2],
    [3,4]
])

# get the eigenvalues
evals = np.linalg.eig(matrix)[0]
```

The two eigenvalues (rounded to the nearest hundredth) are -0.37 and 5.37 .

But the important question isn't *which function returns the eigenvalues*; instead, the important question is *how are the eigenvalues of a matrix identified*?

To find the eigenvalues of a matrix, we start with the eigenvalue equation shown in Equation 13-1 and do some simple arithmetic, as shown in Equation 13-2.

Equation 13-2. Eigenvalue equation, reorganized

$$\begin{aligned}\mathbf{A}\mathbf{v} &= \lambda\mathbf{v} \\ \mathbf{A}\mathbf{v} - \lambda\mathbf{v} &= \mathbf{0} \\ (\mathbf{A} - \lambda\mathbf{I})\mathbf{v} &= \mathbf{0}\end{aligned}$$

The first equation is an exact repeat of the eigenvalue equation. In the second equation, we simply subtracted the right-hand side to set the equation to the zeros vector.

The transition from the second to the third equation requires some explanation. The left-hand side of the second equation has two vector terms, both of which involve $\mathbf{v}$. So we factor out the vector. But that leaves us with the subtraction of a matrix and a scalar ($\mathbf{A} - \lambda$), which is not a defined operation in linear algebra.³ So instead, we *shift* the matrix by λ . That brings us to the third equation. (Side note: the expression $\lambda\mathbf{I}$ is sometimes called a *scalar matrix*.)

What does that third equation mean? It means that *the eigenvector is in the null space of the matrix shifted by its eigenvalue*.

If it helps you understand the concept of the eigenvector as the null-space vector of the shifted matrix, you can think of adding two additional equations:

$$\begin{aligned}\tilde{\mathbf{A}} &= \mathbf{A} - \lambda\mathbf{I} \\ \tilde{\mathbf{A}}\mathbf{v} &= \mathbf{0}\end{aligned}$$

Why is that statement so insightful? Remember that we ignore trivial solutions in linear algebra, so we do not consider $\mathbf{v} = \mathbf{0}$ to be an eigenvector. And that means that the matrix shifted by its eigenvalue is singular, because only singular matrices have a nontrivial null space.

And what else do we know about singular matrices? We know that their determinant is zero. Hence:

$$|\mathbf{A} - \lambda\mathbf{I}| = 0$$

³ As I wrote in Chapter 5, Python will return a result, but that is the scalar broadcast subtracted, which is not a linear algebra operation.

Believe it or not, that's the key to finding eigenvalues: shift the matrix by the unknown eigenvalue λ , set its determinant to zero, and solve for λ . Let's see how this looks for a 2×2 matrix:

$$\begin{aligned} \left| \begin{bmatrix} a & b \\ c & d \end{bmatrix} - \lambda \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \right| &= 0 \\ \begin{vmatrix} a - \lambda & b \\ c & d - \lambda \end{vmatrix} &= 0 \\ (a - \lambda)(d - \lambda) - bc &= 0 \\ \lambda^2 - (a + d)\lambda + (ad - bc) &= 0 \end{aligned}$$

You could apply the quadratic formula to solve for the two λ values. But the answer itself isn't important; the important thing is to see the logical progression of mathematical concepts established earlier in this book:

- The matrix-vector multiplication acts like scalar-vector multiplication (the eigenvalue equation).
- We set the eigenvalue equation to the zeros vector, and factor out common terms.
- This reveals that the eigenvector is in the null space of the matrix shifted by the eigenvalue. We do not consider the zeros vector to be an eigenvector, which means the shifted matrix is singular.
- Therefore, we set the determinant of the shifted matrix to zero and solve for the unknown eigenvalue.

The determinant of an eigenvalue-shifted matrix set to zero is called the *characteristic polynomial* of the matrix.

Notice that in the previous example, we started with a 2×2 matrix and got a λ^2 term, which means this is a second-order polynomial equation. You might remember from your high-school algebra class that an n th order polynomial has n solutions, some of which might be complex-valued (this is called the fundamental theorem of algebra). So, there will be two values of λ that can satisfy the equation.

The matching 2s are no coincidence: the characteristic polynomial of an $M \times M$ matrix will have a λ^M term. That is the reason why an $M \times M$ matrix will have M eigenvalues.